IR ALÉM 1 | COMUNICAÇÃO E AUTOMAÇÃO

REST e e-mail simulado

Relatório do fluxo HTTP, das regras e da validação, com reprodução guiada.

Grupo Aura | Murilo Salla - RM568041
Elias da Silva de Souza - RM568500 | Julia Duarte de Carvalho - RM567816
FIAP - Inteligência Artificial - Ano 2 | Tutor: Leonardo Ruiz Orabona
Revisão: 27/10/2026 | Validação guiada: 26/10/2026

1. Fluxo implementado e critérios

Desenvolvemos cliente e serviço Python locais. Enviamos dados por POST /leituras e consultamos até 100 registros por GET /leituras. Validamos o contrato de telemetria e persistimos leitura, episódios e conteúdo de e-mail em uma transação SQLite. Depois exportamos a mensagem .eml. Não utilizamos SMTP ou envio externo.

A demonstração gera cinco leituras sintéticas próprias, identificadas por cardioia-rest-demo e nova sessão por execução. Reutilizamos o contrato da entrega principal; transport informa a origem declarada, serial_demo ou mqtt, enquanto esta cópia trafega por HTTP. O teste dispensa Wokwi e não encaminha MQTT automaticamente para REST.

2. Regras, integridade e limites

- **Entrada:** acima de 120 BPM artificiais ou 30 °C de temperatura ambiente. **Retorno:** até 108 BPM ou 28 °C, por motivo.
- Um novo motivo gera e-mail; permanecer no mesmo episódio não gera outro. Neste desafio basta uma leitura elegível; o painel exige duas.
- Dados recuperados, idade acima de 10 s ou fila maior que uma posição não alteram episódios. Sensor inválido não resolve alerta de temperatura.
- Mesma identidade e conteúdo retornam duplicate: true; conflito ou sequência fora de ordem recebem HTTP 409. Verificamos BPM e tempos.
- Preservamos episódios após reinício e reconstruímos e-mails ausentes a partir do banco. O cliente informa erros HTTP e falhas de conexão.

Atendemos em 127.0.0.1, uma requisição por vez, sem autenticação/TLS ou limite de retenção. Os horários de coleta são informados pelo cliente. Temperatura é ambiente e zero pulsos representa ausência de acionamentos; não realizamos diagnóstico.

3. Resultados observados

Leitura	BPM / ambiente	Resultado
1	72 / 24 °C	Sem alerta ou e-mail.
2	132 / 31 °C	Primeiro episódio: e-mail 1.
3	144 / 32 °C	Episódio mantido, sem nova mensagem.
4	72 / 24 °C	Episódios encerrados.
5	132 / 31 °C	Novo episódio: e-mail 2; reenvio sem duplicar.

Na sessão 9ed8af36461d489c, inspecionamos os dois e-mails e consultamos as cinco leituras após reiniciar o serviço, com registros originais preservados. Conferimos sete testes HTTP do desafio e 23 testes na suíte Python da Fase 3. As evidências ficam em document/other/fase3/evidencias/2026-10-26/rest/validacao_guiada/.

4. Como reproduzir: seguir na ordem

1. Preparar e baixar: instalar Git e Python 3.12 pelos links do guia inicial. No Windows, abrir Iniciar → PowerShell. No Explorador, criar C:\projetos; no terminal, executar uma linha por vez:

```
Set-Location -LiteralPath 'C:\projetos'

git clone https://github.com/murilosalla-blip/fiap-ano02-fase03-cardioia.git

Set-Location -LiteralPath '.\fiap-ano02-fase03-cardioia'

Test-Path .\README.md
```

Esperado: True. Se já há uma cópia, entrar no caminho real dela e pular o clone. Vamos chamar essa janela de **terminal A**. Não executar os comandos no navegador.

2. Criar o ambiente e instalar: criar .venv somente se não existir. Se python não indicar 3.12, conferir a instalação antes de seguir.

```
python -m venv .venv

.\.venv\Scripts\python.exe -m pip install -r config/fase3/requirements-mqtt.txt
```

3. Iniciar a API no terminal A: executar a linha abaixo e manter essa janela aberta. Esperar a mensagem API local: http://127.0.0.1:8765.

```
.\.venv\Scripts\python.exe src/fase3/python/servico_rest.py
```

4. Verificar no navegador: abrir /saude e esperar status: ok. Abrir /leituras; antes do primeiro envio, a lista estará vazia. A raiz / retorna rota inexistente. Esta API apresenta texto JSON, não gráficos.

5. Enviar no terminal B: abrir outro PowerShell e entrar na mesma pasta com Set-Location -LiteralPath e o caminho real entre aspas. Executar uma vez:

```
.\.venv\Scripts\python.exe src/fase3/python/cliente_rest.py --demonstrar
```

Esperado: cinco leituras em received, dois identificadores de e-mail e duplicate: true em repeated. Guardar session_id. Cada nova demonstração cria outra sessão e dois e-mails; seus números podem variar.

6. Consultar no terminal B: executar abaixo. Esperar a mesma sessão, sem gerar dados novos.

```
.\.venv\Scripts\python.exe src/fase3/python/cliente_rest.py --consultar
```

7. Abrir e-mails: no Explorador, abrir .tools/runtime/fase3/rest/emails dentro do projeto. Abrir os .eml das leituras 2 e 5 em leitor de e-mail ou Bloco de Notas. Conferir [SIMULACAO], sessão, motivos, 132 BPM e 31 °C ambiente. Os destinatários example.invalid são fictícios.

8. Conferir persistência: no A, pressionar Ctrl+C e repetir o comando do passo 3. No B, repetir somente o passo 6. Esperar as cinco leituras originais; não executar --demonstrar neste teste.

9. Encerrar: pressionar Ctrl+C no A. Preservar a pasta .tools/runtime/fase3/rest. Se qualquer comando falhar, parar e guardar sua saída. O guia REST com diagnóstico detalha cada conferência. No Linux/macOS, usar python3.12 para criar o ambiente, cd e .venv/bin/python.